Course Management System – Project Report

Course Name: PCCCS495 – Term II Project

Project Title: Course Management System

Student Name: Debapriya Mondal

Roll Number: 12024052017042

Session: Spring 2026

1. Introduction

Educational institutions often rely on manual processes or disconnected systems to manage course registration, teaching materials, and student progress. These approaches create operational challenges such as difficulty registering students, verifying prerequisites, and monitoring course capacity.

This project implements a Java-based desktop Course Management System that supports role-based login, course registration, weekly progress tracking, and quizzes. The system demonstrates core Object-Oriented Programming (OOP) concepts while providing a practical academic management solution.

2. Problem Statement

Many institutions rely on spreadsheets or rigid software systems to manage courses. This leads to poor visibility of student progress and difficulty managing enrollment.

Key challenges include:

- Hardcoded user data
- Lack of centralized progress tracking
- Difficulty maintaining enrollment records
- Limited teacher visibility of student activity

3. System Design

3.1 Architecture Overview

The system follows a layered architecture consisting of UI, service, model, learning utility, and quiz layers. This separation improves maintainability and modularity.

Main layers:

- UI Layer – Handles user interaction
- Service Layer – Contains business logic
- Model Layer – Represents domain objects
- Learning Layer – Stores progress data
- Quiz Layer – Manages weekly quizzes

4. Object-Oriented Programming Concepts

4.1 Abstraction

Abstraction hides complex logic behind simple service methods. The user interface calls a method without needing to know the internal validation logic.

Example Code:

```
public RegistrationResult registerStudentForCourse(String studentId,
String courseId) {
    Student student = students.get(studentId);

    if (student == null) {
        return RegistrationResult.failure("Student not found");
    }

    // Additional validation logic
}
```

Explanation: The UI simply calls registerStudentForCourse(). The method internally checks if the student exists, verifies prerequisites, and validates course capacity.

4.2 Inheritance

Inheritance allows UI classes to reuse functionality from Swing components.

Example Code:

```
public class LoginDialog extends JDialog {

    private AuthenticationManager authManager;

    public LoginDialog(AuthenticationManager authManager) {
        this.authManager = authManager;
    }
}
```

Explanation: LoginDialog inherits all dialog functionality from JDialog such as window display, event handling, and rendering.

4.3 Polymorphism

Polymorphism allows objects to define their own behavior for shared methods.

Example Code:

```
@Override
public String toString() {
    return studentId + " - " + name;
}
```

Explanation: The toString() method is overridden to provide a meaningful text representation of the student object.

4.4 Exception Handling

Example Code:

```
private void loadUsersFromCSV() {
    try (BufferedReader br = new BufferedReader(new
FileReader("users.csv"))) {

        String line;
        while ((line = br.readLine()) != null) {
            String[] values = line.split(",");
        }

    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

Explanation: The try-with-resources statement ensures that the file is automatically closed after reading. If an error occurs, it is caught and handled.

5. Implementation Highlights

CSV-Based Authentication

Users are loaded dynamically from a CSV file rather than hardcoding credentials.

Example CSV Format:

```
username,password,role
S001,pass123,STUDENT
T001,admin123,TEACHER
```

Explanation: This allows administrators to add new users without modifying the program.

6. Testing and Error Handling

Test cases performed:

- Valid student login
- Invalid login credentials
- Course registration capacity validation

- Weekly progress update

7. Git Workflow

The project development was managed using Git version control. The development process progressed incrementally from UI implementation to authentication, course registration, and quiz integration.

8. Conclusion and Future Scope

The Course Management System demonstrates how object-oriented programming concepts can be applied to develop a modular academic software system.

Future improvements:

- Move course data to database
- Implement password encryption
- Add analytics dashboard

9. Appendix

Build Commands:

```
Compile:
javac -d bin Main.java src/com/course/registration/model/*.java
src/com/course/registration/service/*.java
src/com/course/registration/ui/*.java

Run:
java -cp bin com.course.registration.Main
```